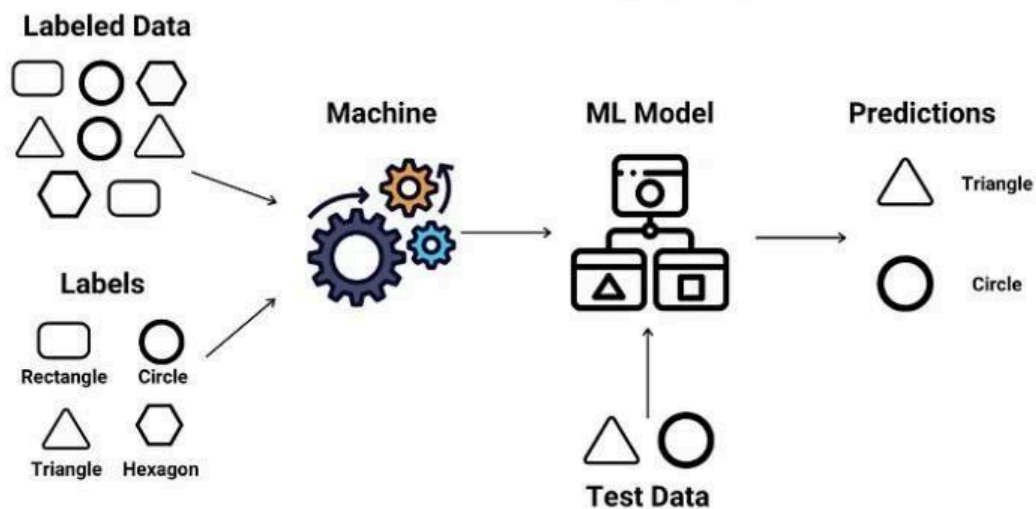


Supervised Learning-Regression

Introduction to Regression analysis

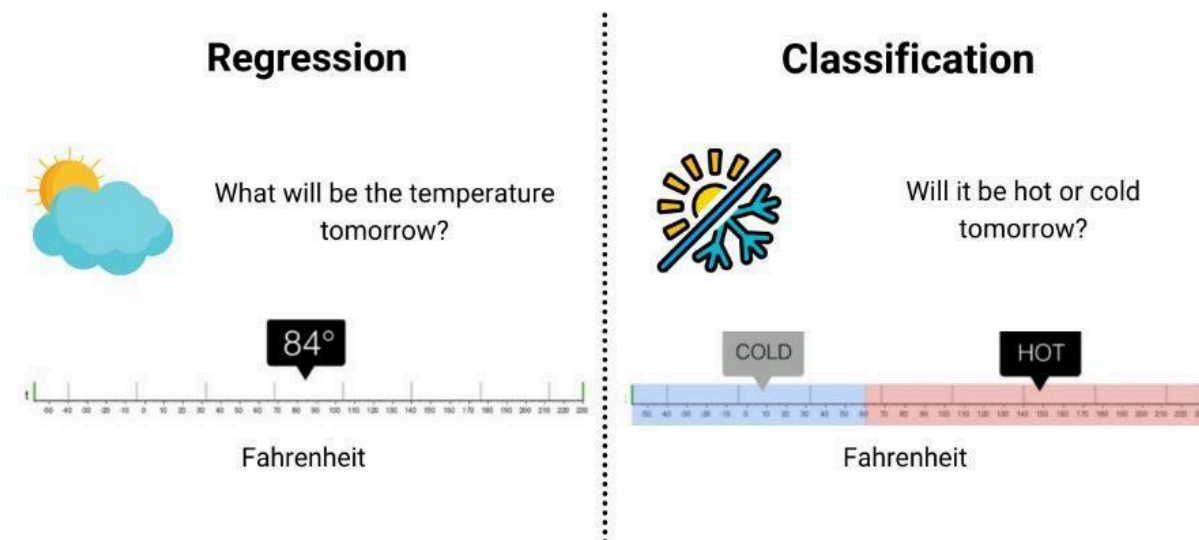
Supervised Learning

Supervised learning is a type of machine learning where an algorithm is trained on labeled data. The model learns from input-output pairs, where the correct output is already known. The goal is to predict the output for new, unseen data.



There are two main types of supervised learning:

1. **Classification:** Predicts discrete labels (e.g., spam or not spam).
2. **Regression:** Predicts continuous values (e.g., house prices).



Classification

Classification models classify our outputs to certain categories. If the number of categories are only two then it is specially called binary classification. For greater number of categories it is called multi-class classification.

Some examples are:

- *Whether a patient has cancer or not*

Regression

Regression models are for labeling outputs with continuous values.

- Predicting house prices, or
- How long is it gonna take you to get home

are both for regression models because the results are ever changing.

The key components of supervised learning include:

1. **Input Features (X):** These are the independent variables or attributes used by the model to make predictions. They could be anything from numerical data, such as age or income, to more complex data like images or text.
2. **Output Labels (Y):** These are the dependent variables or target outcomes that the model is trying to predict. In classification tasks, the outputs are discrete labels (e.g., "dog" or "cat"), while in regression tasks, the outputs are continuous values (e.g., house prices).

3. **Training Data:** A labeled dataset that the model uses to learn. Each example in the dataset consists of an input (X) and its corresponding correct output (Y).
4. **Learning Algorithm:** This is the method the model uses to learn the relationship between inputs and outputs.

Introduction to Regression Analysis:

Regression analysis is a key technique in supervised learning used to predict continuous outcomes. It helps in understanding the relationship between a dependent variable (the outcome) and one or more independent variables (the inputs or predictors). The primary goal of regression is to create a model that can predict the value of the dependent variable given the independent variables.

There are various types of regression techniques, with the most common being:

1. **Linear Regression:** Models the relationship between the dependent variable (Y) and one or more independent variables (X) by fitting a straight line ($Y = bX + c$).
2. **Multiple Regression:** Extends linear regression to multiple input variables.
3. **Polynomial Regression:** Fits a curve to the data instead of a straight line.
4. **Logistic Regression:** Though called regression, it's used for classification tasks, predicting the probability of an event.

Simple Linear Regression:

Simple linear regression is a model that assesses the relationship between a dependent variable and an independent variable.

The simple linear model is expressed using the following equation:

$$\mathbf{y=mx+b}$$

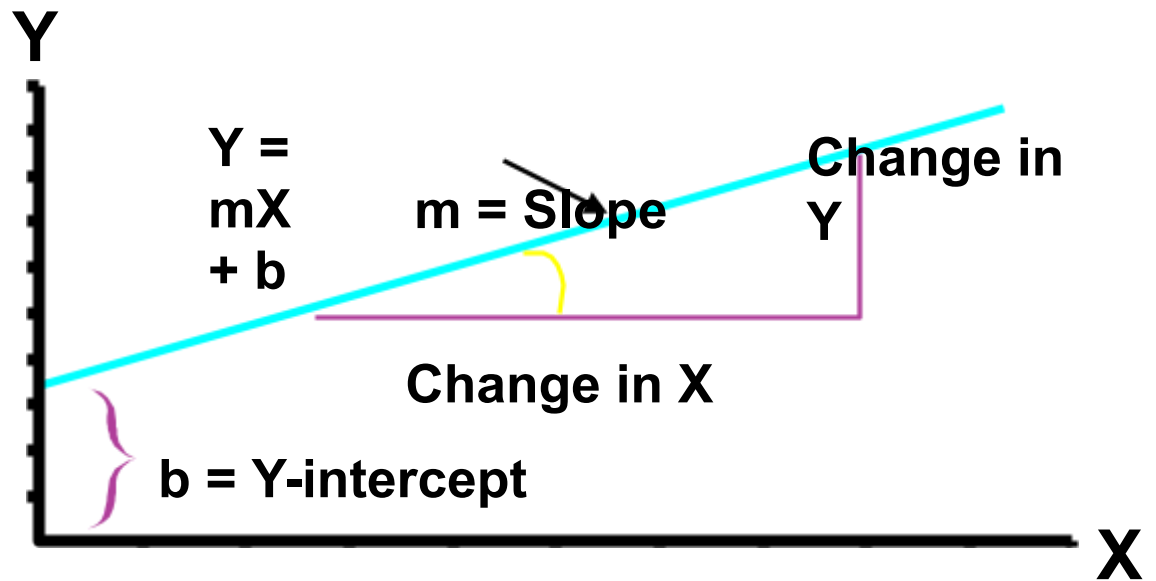
where,

y - Dependent variable

x - Independent variable

m - Slope

b - Intercept



Example: Suppose there is a marketing company A, who does various advertisement every year and get sales on that. The below list shows the advertisement made by the company in the last 5 years and the corresponding sales:

Advertisement	Sales
\$90	\$1000
\$120	\$1300
\$150	\$1800
\$100	\$1200
\$130	\$1380
\$200	??

Supervised Learning-Regression

Measure of linear relationship, Regression with stats models

Regression algorithm (using machine learning) and **regression with statistical models** lies in their primary goals, approaches, and outputs. Both aim to model relationships between variables, but their use cases and how they handle data can be quite distinct.

- **Regression Algorithm (Machine Learning):** The focus is on **prediction** and **generalization** to unseen data. It aims to create a model that can predict the target (dependent variable) based on the features (independent variables).
- **Regression with Stats (Statistical Models):** The focus is on **understanding relationships** between variables. It's often used for explaining the nature of the relationship between independent variables (predictors) and the dependent variable (outcome).

Aspect	Regression Algorithm (Machine Learning)	Regression with Stats (Statistical Models)
Focus	Prediction and generalization	Understanding relationships and inference
Goal	Maximize prediction accuracy	Statistical significance, hypothesis testing
Output	Predictions, error metrics (MSE, R^2)	Coefficients, p-values, confidence intervals
Data Handling	Focus on large datasets, no assumptions	Focus on assumptions and model diagnostics

To measure the linear relationship between two variables and perform regression analysis using **statsmodels** in Python, you can follow these steps:

1. Measure Linear Relationship (Correlation)

Before performing regression, it's common to measure the linear relationship between two variables using **correlation**. You can use **Pearson's correlation coefficient** to measure this.

python

```
import pandas as pd
import numpy as np

# Example data
data = {'X': [1, 2, 3, 4, 5], 'Y': [2, 4, 5, 4, 5]}
df = pd.DataFrame(data)

# Pearson correlation coefficient
correlation = df['X'].corr(df['Y'])
print("Pearson Correlation Coefficient:", correlation)
```

Pearson Correlation Coefficient: 0.7745966692414834

The correlation coefficient ranges from -1 to 1:

- 1 indicates a perfect positive linear relationship.
- -1 indicates a perfect negative linear relationship.
- 0 indicates no linear relationship.

2. Linear Regression Using Statsmodels

You can use the **statsmodels** library to perform linear regression. Here's an example:

Step-by-step guide:

1. Install **statsmodels** if not already installed.

```
pip install statsmodels
```

2. Perform the regression:

python

```
import statsmodels.api as sm

# Define the dependent (Y) and independent (X) variables
X = df['X'] # Independent variable
Y = df['Y'] # Dependent variable

# Add a constant to the independent variable (intercept)
X = sm.add_constant(X)

# Fit the regression model
model = sm.OLS(Y, X).fit()

# Print out the summary of the regression
print(model.summary())
```

- **sm.add_constant(X)** adds a constant term to the predictor variable for the intercept.
- **sm.OLS(Y, X)** creates the OLS (Ordinary Least Squares) model.
- **model.fit()** fits the model to the data.
- **model.summary()** provides a detailed statistical summary, including coefficients, R-squared value, p-values, and confidence intervals.

```

                        OLS Regression Results
=====
Dep. Variable:          Y      R-squared:          0.600
Model:                  OLS    Adj. R-squared:      0.467
Method:                 Least Squares    F-statistic:      4.500
Date:                  Thu, 17 Oct 2024    Prob (F-statistic): 0.124
Time:                  09:20:58    Log-Likelihood:    -5.2598
No. Observations:      5      AIC:              14.52
Df Residuals:          3      BIC:              13.74
Df Model:               1
Covariance Type:       nonrobust
=====
                        coef    std err          t      P>|t|      [0.025      0.975]
-----
const                2.2000     0.938     2.345     0.101     -0.785     5.185
X                    0.6000     0.283     2.121     0.124     -0.300     1.500
=====
Omnibus:              nan    Durbin-Watson:      2.017
Prob(Omnibus):         nan    Jarque-Bera (JB):    0.570
Skew:                  0.289    Prob(JB):            0.752
Kurtosis:              1.450    Cond. No.            8.37
=====
```


3. Interpreting the Results

- **R-squared:** Indicates how well the independent variable explains the variability in the dependent variable.
- **p-value:** Tests the null hypothesis that the coefficient of a variable is zero (no relationship). A p-value less than 0.05 usually indicates statistical significance.
- **Coefficients:** The slope of the regression line (relationship strength) and the intercept.

Studying regression with **statsmodels** offers several advantages, especially for those who want a deeper understanding of regression analysis and statistical modeling. Here are some reasons why **statsmodels** is beneficial for studying regression:

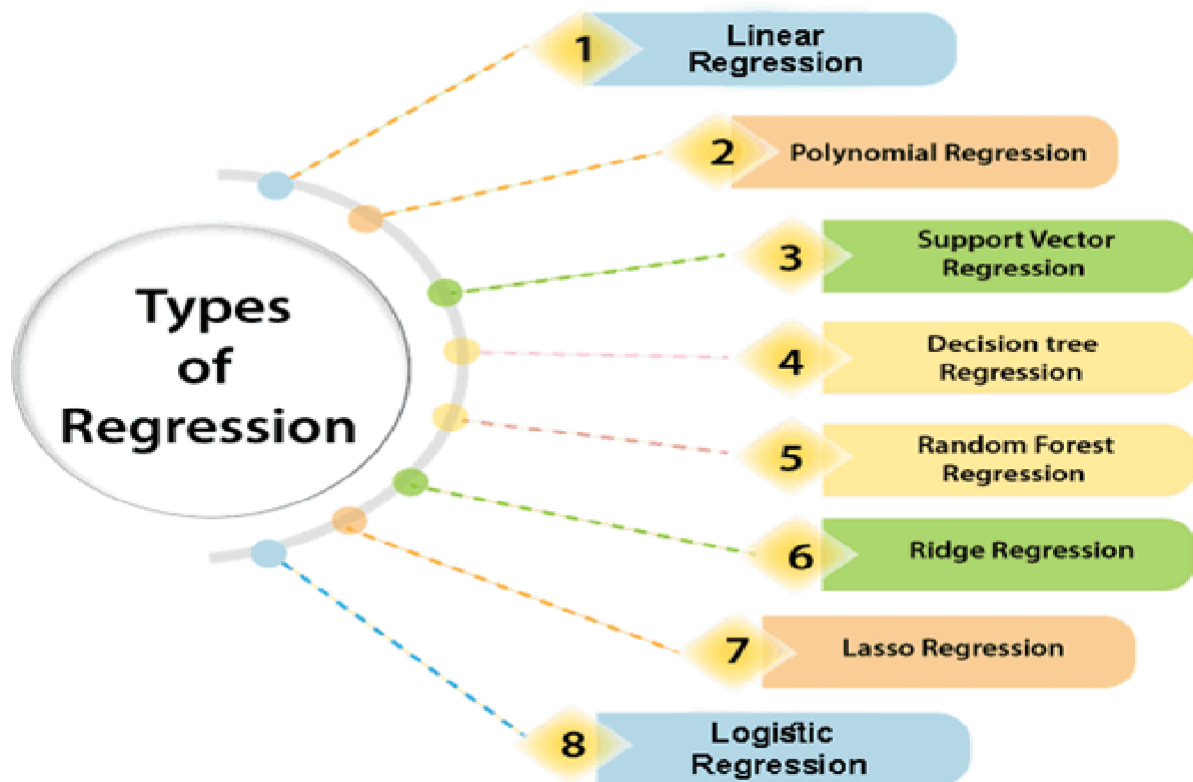
Detailed Statistical Output

Unlike other libraries like **scikit-learn**, which focus primarily on prediction, **statsmodels** provides comprehensive statistical information about the regression model. The output includes:

- **R-squared and Adjusted R-squared:** Measures of model fit.
- **P-values:** Help determine the statistical significance of predictors.
- **Confidence Intervals:** Shows the range of values for the coefficients with a certain level of confidence.

Supervised Learning-Regression

Types of regression



1. Linear Regression

The most extensively used modelling technique is linear regression, which assumes a linear connection between a dependent variable (Y) and an independent variable (X).

It employs a regression line, also known as a best-fit line.

The linear connection is defined as $Y = c + m \cdot X + e$

Where,

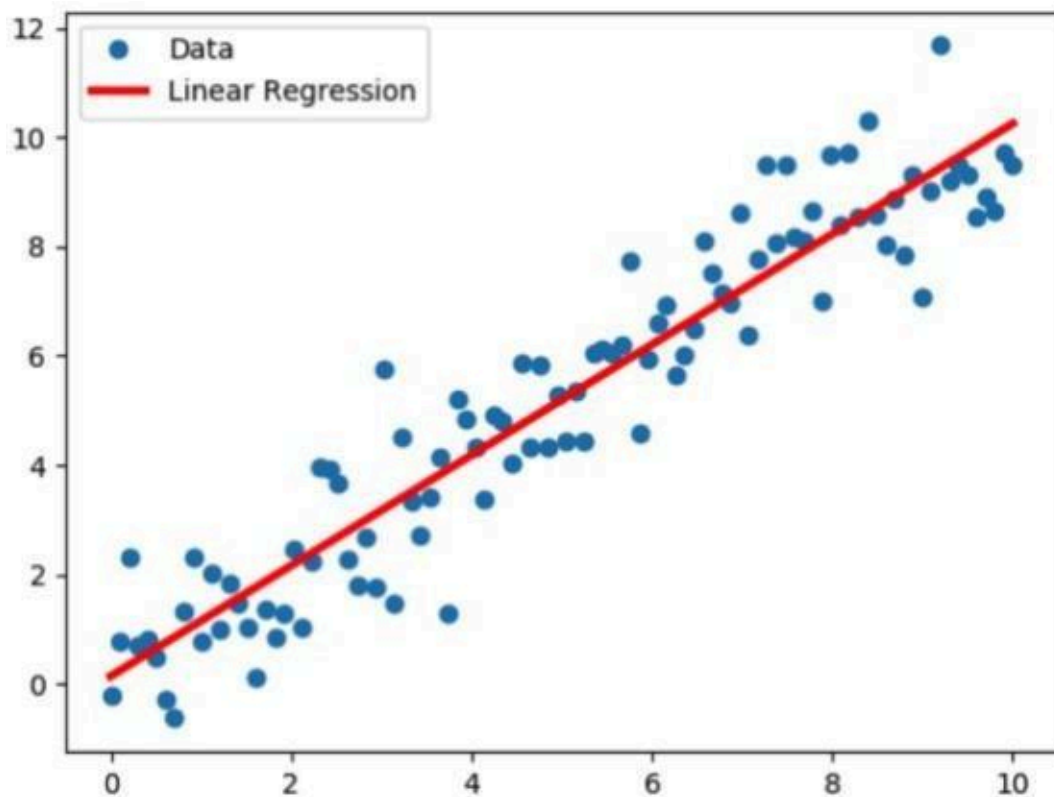
‘c’ denotes the intercept,

‘m’ denotes the slope of the line, and

‘e’ is the error term.

The linear regression model can be simple (with only one dependent and one independent variable) or complex (with numerous dependent and independent

variables) (with one dependent variable and more than one independent variable).



$$Y = c + m \cdot X + e$$

The best fit line is determined by varying the values of m and c .

The predictor error is the difference between the observed values and the predicted value.

The values of m and c get selected in such a way that it gives the minimum predictor error.

There are different types of linear regression. The two major types of linear regression are

- simple linear regression and
- multiple linear regression.

For a single variable linear regression: $Y = \beta_0 + \beta_1 X + \epsilon$

For multiple variables (multiple linear regression): $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \epsilon$

Where:

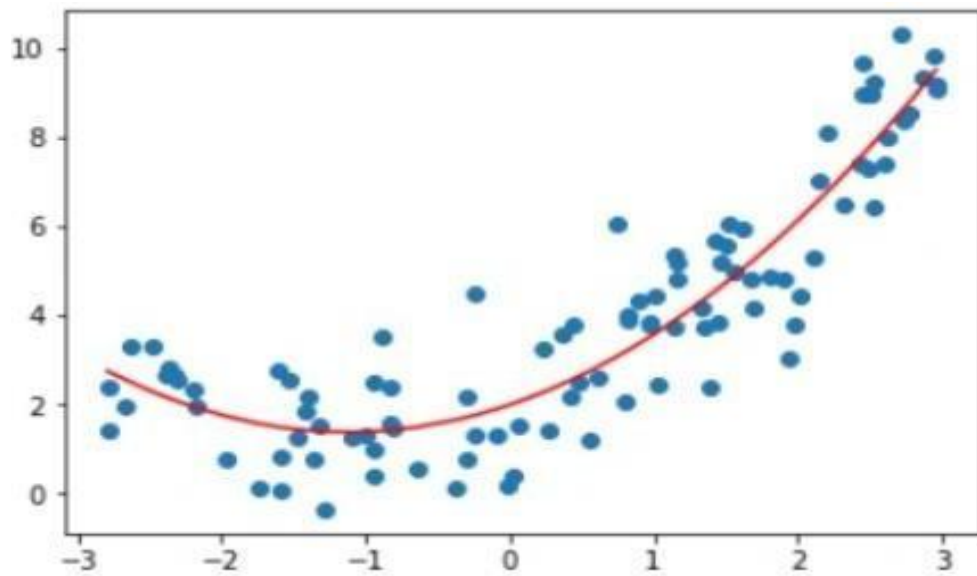
- Y is the dependent variable (what you are predicting),
- $X_1, X_2, \dots, X_n$ are the independent variables (predictors),
- β_0 is the intercept (value of Y when all predictors are 0),
- $\beta_1, \dots, \beta_n$ are the coefficients for the predictors,
- ϵ is the error term.

2. Polynomial Regression

Polynomial regression is an extension of linear regression where the relationship between the dependent variable and the independent variables is modeled as a polynomial. This allows for **non-linear relationships** between the variables but still uses the framework of linear regression to estimate the model coefficients.

For a degree d polynomial: $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \dots + \beta_d X^d + \epsilon$ Where:

- $X^2, X^3, \dots, X^d$ are higher-degree terms.
- d is the degree of the polynomial.



Suppose you are predicting the **profit** of a company based on its **advertising budget**. The relationship might not be linear, but quadratic or cubic, meaning the return on investment decreases after a certain point.

- Feature X : Advertising budget.
- Target Y : Company profit.

The model could be quadratic, like: $\text{Profit} = \beta_0 + \beta_1 \times \text{Budget} + \beta_2 \times \text{Budget}^2 + \epsilon$

Use Case:

- Predicting the trajectory of a projectile where the height is a function of time squared.
- Modeling economic variables where relationships between variables are nonlinear.

Supervised Learning-Regression

Simple Linear Regression using sample dataset.

Simple Linear Regression is a statistical technique that models the relationship between two variables: a dependent variable (Y) and an independent variable (X). It assumes that the relationship between the variables is linear, which means that we can represent it with a straight line of the form:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

Where:

- Y is the dependent variable (what you are trying to predict).
- X is the independent variable (the predictor).
- β_0 is the intercept of the regression line (the value of Y when X = 0).
- β_1 is the slope of the regression line (how much Y changes for a unit change in X).
- ϵ is the error term (captures any deviations from the exact linear relationship).

Methodology

1. **Hypothesis Setup:** Simple linear regression aims to find the line that best fits the data points in a two-dimensional plane.
2. **Data Collection:** Collect data where you have observations for both the dependent and independent variables.
3. **Fitting the Model:** The goal is to estimate the parameters β_0 & β_1 such that the sum of squared residuals (the difference between the actual value and the predicted value) is minimized.
4. **Model Evaluation:** After fitting the model, evaluate how well the model predicts by using metrics such as **R-squared** and the **mean squared error (MSE)**.
5. **Prediction:** Once the model is trained, we can use the estimated coefficients to predict new data.

Use Case: Predicting House Prices

Consider a real-world scenario where you want to predict house prices (Y) based on the size of the house (X). This is a simple linear regression problem because house price can be predicted using one feature: house size.

Dataset

For simplicity, let's use a sample dataset of house sizes (in square feet) and house prices (in thousands of dollars):

Size (sq.ft)	Price (thousands of dollars)
750	150
800	160
850	180
900	200
1000	220

- **Import Libraries:** The necessary libraries to be imported (numpy, matplotlib, and sklearn).
- **Define the Dataset:** The initial dataset of house sizes and prices
- **Model Creation:** A LinearRegression object is created, and the model is trained on the existing dataset using the .fit() method.
- **New Data for Prediction:** A new dataset (new_house_sizes) is created with house sizes of 1100, 1200, and 1300 square feet. This data will be used to make new price predictions using the .predict() method.
- **Plotting:** The plot includes the original data points, the regression line, and the predicted prices for the new house sizes (displayed in green).

```

# Step 1: Import necessary libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Step 2: Define the dataset
# House size (independent variable X)
X = np.array([750, 800, 850, 900, 1000]).reshape(-1, 1)

# House prices (dependent variable Y)
Y = np.array([150, 160, 180, 200, 220])

```

- `numpy (np)`: Used for handling arrays and reshaping data.
- `matplotlib.pyplot (plt)`: Used for plotting graphs, to visualize the relationship between house size and price.
- `LinearRegression`: The regression model from the **scikit-learn** library.
- `X`: This array contains the sizes of houses in square feet. The `reshape(-1, 1)` converts the 1D array into a 2D array as required by the regression model. Each row represents a different house size.
- `Y`: This array contains the corresponding prices (in thousands of dollars) for the house sizes in `X`.

```

# Step 3: Create a linear regression model and fit the data
model = LinearRegression()
model.fit(X, Y)

# Step 4: Make predictions using the model on existing data
Y_pred = model.predict(X)

```

- `LinearRegression()` : Initializes a linear regression model.
- `model.fit(X, Y)` : Fits (or trains) the model using the data provided (`X` as input and `Y` as output). The model learns the relationship between house size and price, finding the best-fitting line.
- `model.predict(X)` : Predicts the house prices for the given house sizes (`X`) using the trained model.
- `Y_pred` : Stores the predicted prices for the house sizes in `X` (original data). These are the values lying on the regression line.

```
# Step 5: New data for prediction
new_house_sizes = np.array([1100, 1200, 1300]).reshape(-1, 1) # New house sizes (in sq.ft)

# Step 6: Predict prices for new house sizes
predicted_prices = model.predict(new_house_sizes)

# Step 7: Display results
print("Predicted Prices for New House Sizes:")
for size, price in zip(new_house_sizes, predicted_prices):
    print(f"Size: {size[0]} sq.ft -> Predicted Price: {price:.2f} thousands of dollars")
```

- `new_house_sizes` : This array contains the new house sizes (1100 sq.ft, 1200 sq.ft, and 1300 sq.ft) for which we want to predict the prices. Again, it's reshaped into a 2D array for input into the regression model.
- `model.predict(new_house_sizes)` : Uses the trained model to predict the prices of houses based on the new house sizes provided.
- `predicted_prices` : Stores the predicted prices for the new house sizes.

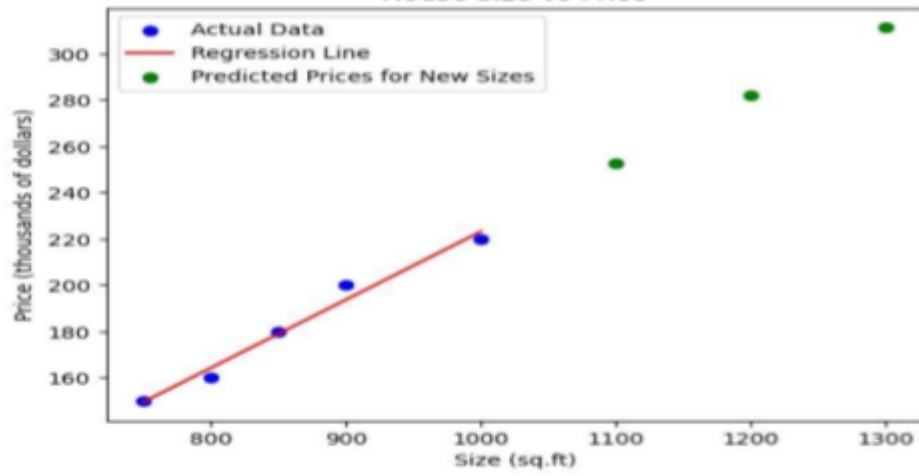
- `for size, price in zip(new_house_sizes, predicted_prices)` : Loops over the new house sizes and their corresponding predicted prices.
- `print()` : Displays the predicted prices for each of the new house sizes, formatted to two decimal places.

```
# Step 8: Plotting the data and regression line
plt.scatter(X, Y, color="blue", label="Actual Data")
plt.plot(X, Y_pred, color="red", label="Regression Line")
plt.scatter(new_house_sizes, predicted_prices, color="green", label="Predicted Prices for New Sizes")
plt.title("House Size vs Price")
plt.xlabel("Size (sq.ft)")
plt.ylabel("Price (thousands of dollars)")
plt.legend()
plt.show()
```

- `plt.scatter(X, Y, color="blue")` : Plots the original data points (house sizes and actual prices) in blue.
- `plt.plot(X, Y_pred, color="red")` : Plots the regression line, which represents the predicted prices based on house size. It shows the best-fit linear relationship between size and price for the original data.
- `plt.scatter(new_house_sizes, predicted_prices, color="green")` : Plots the predicted prices for the new house sizes in green.
- `plt.title()`, `plt.xlabel()`, `plt.ylabel()` : Add a title and labels to the graph for clarity.
- `plt.legend()` : Adds a legend to distinguish between the actual data, the regression line, and the new predictions.
- `plt.show()` : Displays the plot.

Predicted Prices for New House Sizes:
 Size: 1100 sq.ft -> Predicted Price: 252.70 thousands of dollars
 Size: 1200 sq.ft -> Predicted Price: 282.16 thousands of dollars
 Size: 1300 sq.ft -> Predicted Price: 311.62 thousands of dollars

House Size vs Price



Supervised Learning-Regression

Multiple Linear Regression using sample dataset

Multiple Linear Regression is an extension of Simple Linear Regression where we model the relationship between a dependent variable (Y) and multiple independent variables (X1, X2, ..., Xn). The goal is to predict the value of the dependent variable based on several features.

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \epsilon$$

Where:

- Y is the dependent variable (what you are trying to predict).
- X1,X2,...,Xn are the independent variables (predictors).
- β_0 is the intercept.
- $\beta_1, \beta_2, \dots, \beta_n$ are the coefficients for the independent variables.
- ϵ is the error term (captures deviations from the predicted linear relationship).

Use Case: Predicting House Prices with Multiple Features

Consider a real-world example where we want to predict house prices (Y) based on multiple factors such as:

- Size of the house (X1)
- Number of bedrooms (X2)
- Age of the house (X3)

These factors influence the price of the house, and by using multiple linear regression, we can predict prices more accurately than using just one factor.

Sample Dataset

Let's assume the following dataset with three independent variables:

Size (sq.ft)	Bedrooms	Age (years)	Price (thousands of dollars)
750	2	10	150
800	3	8	160
850	3	5	180
900	4	2	200
1000	4	1	220

```
# Step 1: Import necessary libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
```

- `numpy`: Used for creating and handling arrays (e.g., the dataset for the house sizes, bedrooms, and age).
- `matplotlib.pyplot`: Used for plotting graphs to visualize data.
- `sklearn.linear_model.LinearRegression`: A machine learning model for performing linear regression (both simple and multiple regression).

```
# Step 2: Define the dataset
# Independent variables (Size in sq.ft, Bedrooms, Age in years)
X = np.array([
    [750, 2, 10],
    [800, 3, 8],
    [850, 3, 5],
    [900, 4, 2],
    [1000, 4, 1]
])

# Dependent variable (Price in thousands of dollars)
Y = np.array([150, 160, 180, 200, 220])
```


- **Independent Variables (X):**

- Here, `x` represents the features or predictors. It is a 2D array, where each row contains three values:
 - Size of the house in square feet (1st column).
 - Number of bedrooms (2nd column).
 - Age of the house in years (3rd column).
- Example: `[750, 2, 10]` represents a house with 750 sq.ft, 2 bedrooms, and 10 years old.

- **Dependent Variable (Y):**

- `y` is the target variable, which represents the house prices (in thousands of dollars).
- Example: 150 represents a house price of \$150,000.

```
# Step 3: Create a multiple linear regression model and fit the data
model = LinearRegression()
model.fit(X, Y)
```

- `model = LinearRegression()` : Initializes the Linear Regression model from `sklearn`.
- `model.fit(X, Y)` : This is where the model learns the relationship between the independent variables (size, bedrooms, age) and the dependent variable (price). The model identifies the coefficients for each of the features in `x` and the intercept, forming the regression equation:

Regression Equation:

$$\text{Price} = (\text{coefficient}_1 \times \text{Size}) + (\text{coefficient}_2 \times \text{Bedrooms}) + (\text{coefficient}_3 \times \text{Age}) + \text{intercept}$$

```
# Step 4: Make predictions using the model on existing data
Y_pred = model.predict(X)
```

- `model.predict(X)` : The model uses the learned regression equation to predict prices based on the existing data in `x`.
 - `Y_pred` : This array contains the predicted prices for the houses in the original dataset.
-

```
# Step 5: New data for prediction
new_house_data = np.array([
    [1100, 4, 1], # Size: 1100 sq.ft, 4 bedrooms, 1 year old
    [1200, 5, 0], # Size: 1200 sq.ft, 5 bedrooms, new house
    [950, 3, 5],  # Size: 950 sq.ft, 3 bedrooms, 5 years old
    [1050, 4, 2], # Size: 1050 sq.ft, 4 bedrooms, 2 years old
    [1150, 3, 3], # Size: 1150 sq.ft, 3 bedrooms, 3 years old
    [980, 2, 7]   # Size: 980 sq.ft, 2 bedrooms, 7 years old
])
```

- **New House Data:** This is a 2D array representing new houses with different sizes, bedrooms, and ages. We want to use the trained model to predict the prices for these new houses.

```
# Step 6: Predict prices for new house data
predicted_prices = model.predict(new_house_data)
```

- `model.predict(new_house_data)`: The model takes the new house data as input and uses the learned relationship from the training data to predict the prices for these new houses.
- `predicted_prices`: This array stores the predicted prices for the new house data.

```
# Step 7: Display the predicted prices for the new data
print("Predicted Prices for New Houses:")
for data, price in zip(new_house_data, predicted_prices):
    print(f"Size: {data[0]} sq.ft, Bedrooms: {data[1]}, Age: {data[2]} years -> Predicted Price: {price:.2f} thousands of dollars")
```

- **For loop:**
 - `zip(new_house_data, predicted_prices)` pairs each house data entry with its predicted price.

```
# Step 8: (Optional) Visualizing the regression model with existing and new data

# Plot for existing data
plt.scatter(X[:, 0], Y, color="blue", label="Actual Data (Price vs Size)")
plt.plot(X[:, 0], Y_pred, color="red", label="Predicted Price (Existing Data)")

# Plot for new data
plt.scatter(new_house_data[:, 0], predicted_prices, color="green", marker="x", label="Predicted Price (New Houses)", s=100)

# Labels and title
plt.title("House Size vs Price (Including New Predictions)")
plt.xlabel("Size (sq.ft)")
plt.ylabel("Price (thousands of dollars)")
plt.legend()
plt.show()
```

1. Visualization for New Data:

- We added another **scatter plot** for the **new house data** predictions using `plt.scatter()`.
- The `new_house_data[:, 0]` represents the sizes of the new houses, and `predicted_prices` contains their predicted prices.
- The marker style is changed to "x" and colored green to differentiate it from the original data.

2. New Plot Customization:

- The green "x" markers represent the **predicted prices for new houses**.
- The title and labels are updated to make the plot more informative.



Supervised Learning-Regression

Polynomial Regression using sample dataset

Polynomial Regression is an extension of linear regression where the relationship between the independent and dependent variables is modeled as an n-degree polynomial. It's useful when the data shows a non-linear relationship, but we still want to model it with linear techniques by transforming the input features.

The general form of a polynomial regression model is:

$$y = \beta_0 + \beta_1x + \beta_2x^2 + \beta_3x^3 + \dots + \beta_nx^n + \epsilon$$

where:

- y is the dependent variable.
- x is the independent variable.
- $\beta_0, \beta_1, \beta_2, \dots, \beta_n$ are the coefficients of the polynomial terms.
- ϵ is the error term, accounting for noise in the data.

The degree of the polynomial n , determines the complexity of the model. For example:

- $n=1$: Linear regression (a straight line).
- $n=2$: Quadratic regression (a parabola).
- $n=3$: Cubic regression (a more flexible curve), and so on.

Why Use Polynomial Regression?

Polynomial regression is useful when data shows a non-linear trend that cannot be captured by a simple linear model. Examples include:

- Modeling the growth of populations.
- Predicting stock prices with seasonal trends.
- Tracking temperature fluctuations over time.

It provides a simple way to extend linear regression for capturing such non-linear patterns without resorting to more complex models like neural networks or non- parametric techniques.

Steps for Polynomial Regression

1. Import necessary libraries
2. Define the dataset
3. Transform the features into polynomial features
4. Fit the polynomial regression model
5. Predict new values
6. Visualize the result

Step 1: Import necessary libraries

- **numpy**: For handling array operations.
- **matplotlib.pyplot**: For plotting the graphs.
- **LinearRegression**: The linear regression model from sklearn.
- **PolynomialFeatures**: This will be used to transform the input features into polynomial features.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
```

Step 2: Define the dataset

- **X**: The independent variable (input), representing numbers 1 through 10.
- **Y**: The dependent variable (output), following a quadratic pattern (i.e., $Y=X^2$)

```
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
Y = np.array([1, 4, 9, 16, 25, 36, 49, 64, 81, 100]) # Quadratic data (Y = X^2)
```

Step 3: Transform the features into polynomial features

- **PolynomialFeatures(degree=2)**: Transforms the input features into polynomial features of degree 2. For example, if $X = [x_1]$, it becomes $X_{poly} = [1, x_1, x_1^2]$.

- **fit_transform(X)**: Transforms the input array X into polynomial features.

```
poly_features = PolynomialFeatures(degree=2)
X_poly = poly_features.fit_transform(X)
```

Step 4: Fit the polynomial regression model

- **LinearRegression()**: Initializes the linear regression model.
- **model.fit(X_poly, Y)**: Trains the linear regression model using the polynomial features of X and the dependent variable Y.

```
model = LinearRegression()
model.fit(X_poly, Y)
```

Step 5: Make predictions using the model on the training data

- **model.predict(X_poly)**: Uses the trained polynomial regression model to make predictions based on the original X values transformed into polynomial features.

```
Y_pred = model.predict(X_poly)
```

Step 6: New data for prediction

- **new_data**: Represents new data points (11, 12, 13) that were not part of the original dataset.
- **new_data_poly**: Transforms the new data into polynomial features.
- **new_predictions**: Predicts the Y values for the new data using the polynomial regression model.

```
new_data = np.array([11, 12, 13]).reshape(-1, 1)
new_data_poly = poly_features.transform(new_data)
new_predictions = model.predict(new_data_poly)
```

Step 7: Display the predicted prices for new data

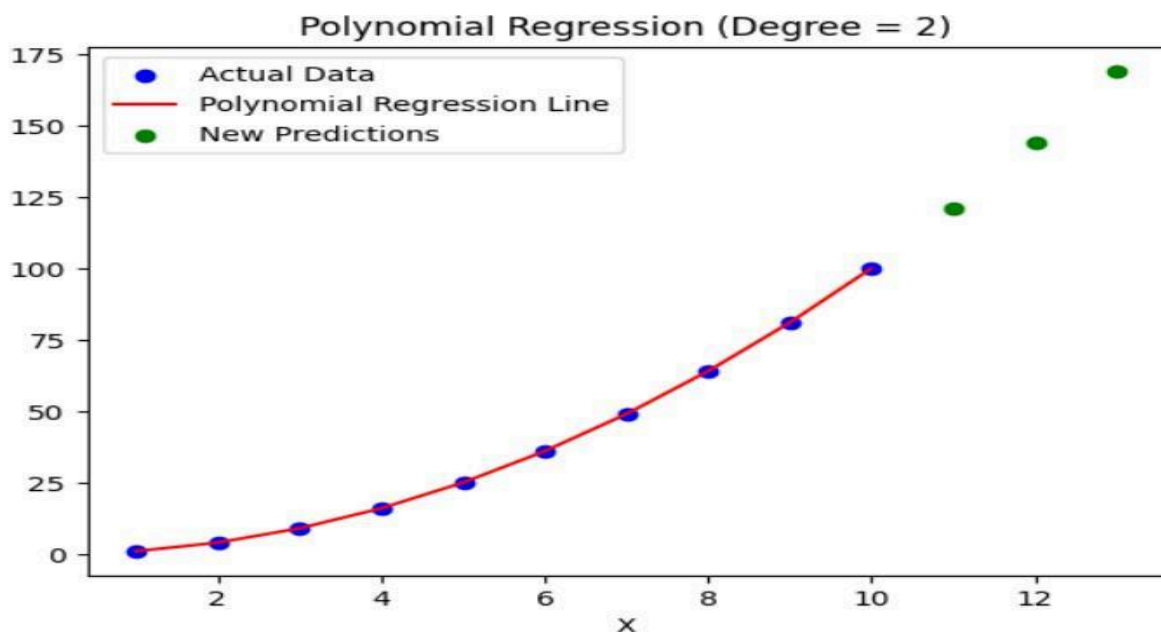
- **zip(new_data, new_predictions)**: Loops through each pair of new data points and their corresponding predicted values.
- **print()**: Displays the predictions for the new data.

```
print("Predicted values for new data:")
for data, price in zip(new_data, new_predictions):
    print(f"X: {data[0]} -> Predicted Y: {price:.2f}")
```

Step 8: Visualization

- **plt.scatter(X, Y, color="blue")**: Plots the original data points.
- **plt.plot(X, Y_pred, color="red")**: Plots the regression line (polynomial curve) based on the predicted values.
- **plt.scatter(new_data, new_predictions, color="green")**: Plots the predicted values for the new data points.
- **plt.show()**: Displays the plot.

```
plt.scatter(X, Y, color="blue", label="Actual Data")
plt.plot(X, Y_pred, color="red", label="Polynomial Regression Line")
plt.scatter(new_data, new_predictions, color="green", label="New Predictions")
plt.title("Polynomial Regression (Degree = 2)")
plt.xlabel("X")
plt.ylabel("Y")
plt.legend()
plt.show()
```



Supervised Learning-Regression

Metrics for Regression

Regression is a type of Machine learning algorithm which helps in finding the relationship between independent and dependent variables.

Before learning about precise metrics, let's familiarize ourselves with a few essential concepts related to regression metrics:

1. True Values and Predicted Values:

In regression, we've got two units of values to compare: the actual target values (authentic values) and the values expected by the model (predicted values). The performance of the model is assessed by means of measuring the similarity among these sets.

2. Evaluation Metrics:

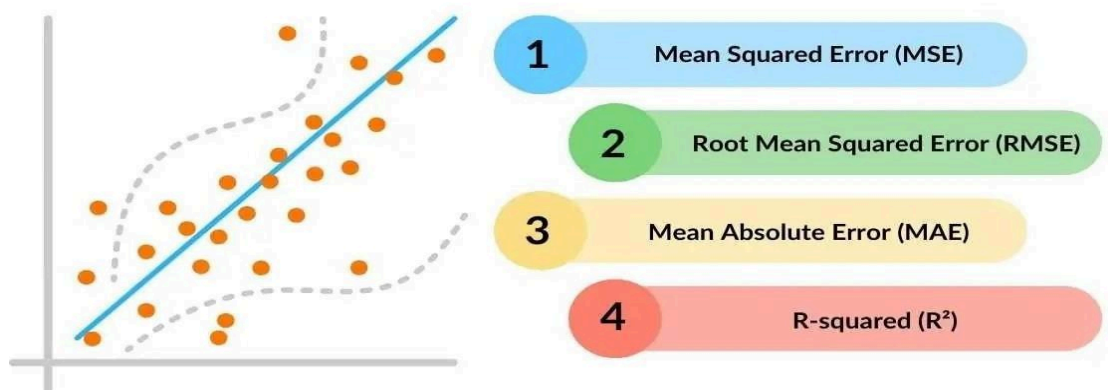
Regression metrics are used to evaluate the performance of regression models by quantifying how well a model's predictions match actual values. Evaluation Metrics for regression are essential for assessing the performance of regression models specifically. These metrics help in measuring how well a regression model is able to predict continuous outcomes.

Types of Regression Metrics

Some common regression metrics in scikit-learn with examples

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R-squared (R^2) Score
- Adjusted R-Squared

4 Common Regression Metrics



1. Mean Absolute Error (MAE)

The diagram shows the formula $MAE = \frac{1}{N} \sum |y - \hat{y}|$. Annotations include: 'Divide by total Number of Data Points' pointing to $\frac{1}{N}$; 'Actual Output' pointing to y ; 'Predicted Output' pointing to $\hat{y}$; 'Sum Of' pointing to the summation symbol $\sum$; and 'Absolute Value of residual' pointing to the absolute value bars $|y - \hat{y}|$.

- **Description:** MAE is the average of the absolute differences between the predicted and actual values. It measures how close predictions are to the actual outcomes.
- **Interpretation:** Lower MAE values indicate better model performance, as it implies smaller errors on average.
- **Sensitivity:** It is less sensitive to outliers than other metrics because it doesn't square the errors. However, it may undervalue large errors.

2. Mean Squared Error (MSE)

$$MSE = \frac{1}{n} \sum \underbrace{(y - \hat{y})^2}_{\text{The square of the difference between actual and predicted}}$$

- **Description:** MSE is the average of the squared differences between actual and predicted values. Squaring the errors penalizes larger errors more than smaller ones.
- **Interpretation:** Lower MSE values are preferred. It emphasizes larger errors, which can be useful if large errors are particularly undesirable in your application.
- **Sensitivity:** Highly sensitive to outliers because errors are squared, causing models with high variance to have higher MSE values.

3. Root Mean Squared Error (RMSE)

The diagram shows the formula $RMSE = \sqrt{MSE}$ and the expanded formula $RMSE = \sqrt{\frac{1}{n} \sum_{j=1}^n (y_j - \hat{y}_j)^2}$.

- **Description:** RMSE is the square root of MSE. It provides the error in the same units as the dependent variable y , making it more interpretable.
- **Interpretation:** Lower RMSE indicates better model performance. It is especially useful when large errors should be heavily penalized.

- **Sensitivity:** Like MSE, it is sensitive to outliers. However, RMSE is more interpretable than MSE because it is in the original units of the outcome variable.

4. R-Squared (Coefficient of Determination)

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

- **Description:** R^2 represents the proportion of variance in the dependent variable explained by the model. It ranges from 0 to 1.
- **Interpretation:** Higher values (closer to 1) indicate that the model explains a large proportion of the variance. An R^2 of 0 means the model does not explain any variance, while 1 indicates perfect prediction.
- **Sensitivity:** Sensitive to data variability. While useful, it does not provide a measure of error magnitude, and its value can be misleading, especially with high-dimensional or overfitted models.

5. Adjusted

R-Squared Formula:

$$\text{Adjusted } R^2 = 1 - (1 - R^2) \frac{n - 1}{n - k - 1}$$

- **Description:** Adjusted R^2 modifies R^2 by adjusting for the number of predictors in the model. This avoids overestimating the model's performance when additional predictors are added.
- **Interpretation:** Higher adjusted R^2 values suggest a better model fit while penalizing for unnecessary predictors.
- **Sensitivity:** Adjusted R^2 is particularly useful when comparing models with different numbers of predictors, as it accounts for added complexity.

Choosing the Right Metric

- **Interpretability:** Metrics like RMSE and MAE provide errors in actual units, making them interpretable.
- **Outliers:** If the data has outliers, MAE or MSLE may be preferable as they are less sensitive than MSE and RMSE.
- **Percentage-Based Accuracy:** Use MAPE or SMAPE for models where percentage accuracy is meaningful and the dataset doesn't contain zero or near-zero values.
- **Model Comparison:** Adjusted R^2 and Explained Variance Score are valuable when comparing models, especially with different numbers of predictors.

Selecting the best metric will depend on your specific goals, data properties, and the application context.

Supervised Learning-Regression

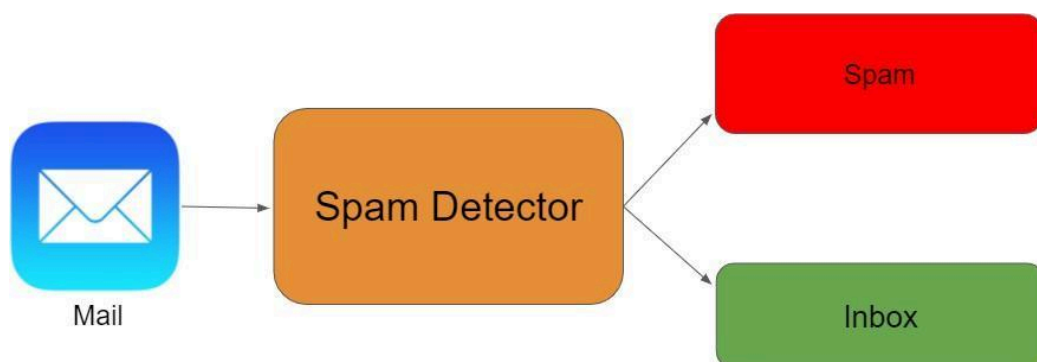
Generalization, Overfitting and Underfitting

Real-world data is inherently complex, encompassing variations, noise, and unpredictable factors. In the realm of machine learning and data science, the ultimate objective is to develop models capable of delivering accurate predictions and valuable insights when confronted with new and unseen data.

To achieve this objective, the concept of generalization plays a pivotal role. Generalization is a widely recognized technique in the world of machine learning and artificial intelligence

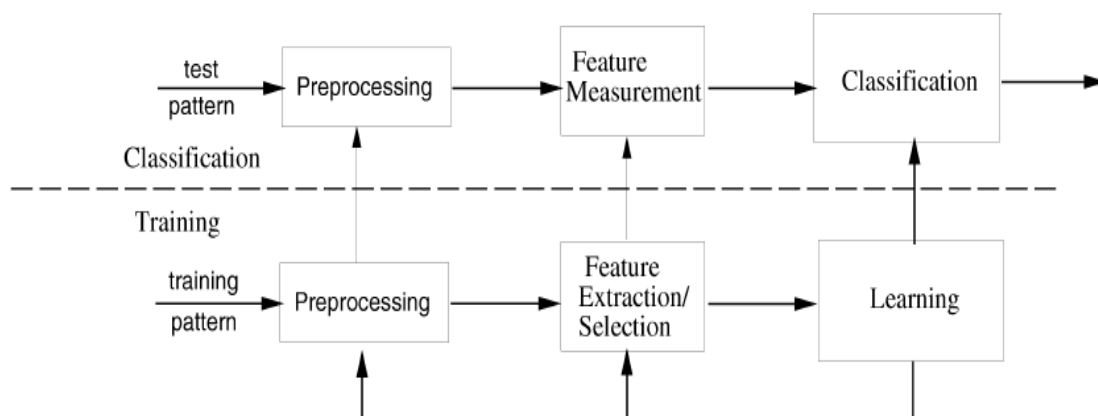
Generalization

- **Definition:** Generalization is the model's ability to perform well on new, unseen data after being trained on a given dataset. A well-generalized model learns patterns in the training data that are relevant to the larger data distribution, allowing it to make accurate predictions on data it hasn't seen.
- **Importance:** Achieving good generalization is essential in machine learning because it enables models to handle real-world scenarios accurately.
- **Influence Factors:** Generalization is influenced by the complexity of the model, the amount and quality of data, and the regularization techniques used.
- A spam email classifier is a great example of generalization in machine learning. Suppose you have a training dataset containing emails labeled as either *spam* or *not spam* and your goal is to build a model that can accurately classify incoming emails as spam or legitimate based on their content.

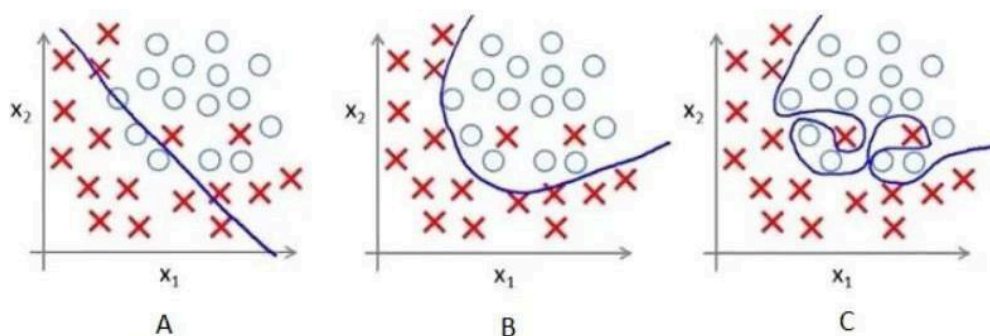


During the training phase, the machine learning algorithm learns from the set of labeled emails, extracting relevant features and patterns to make predictions. The model optimizes its parameters to minimize the training error and achieve high accuracy on the training data.

Now, the true test of the model's effectiveness lies in its ability to generalize to new, unseen emails. When new emails arrive, the model needs to accurately classify them as spam or legitimate without prior exposure to their content. This is where generalization comes in.

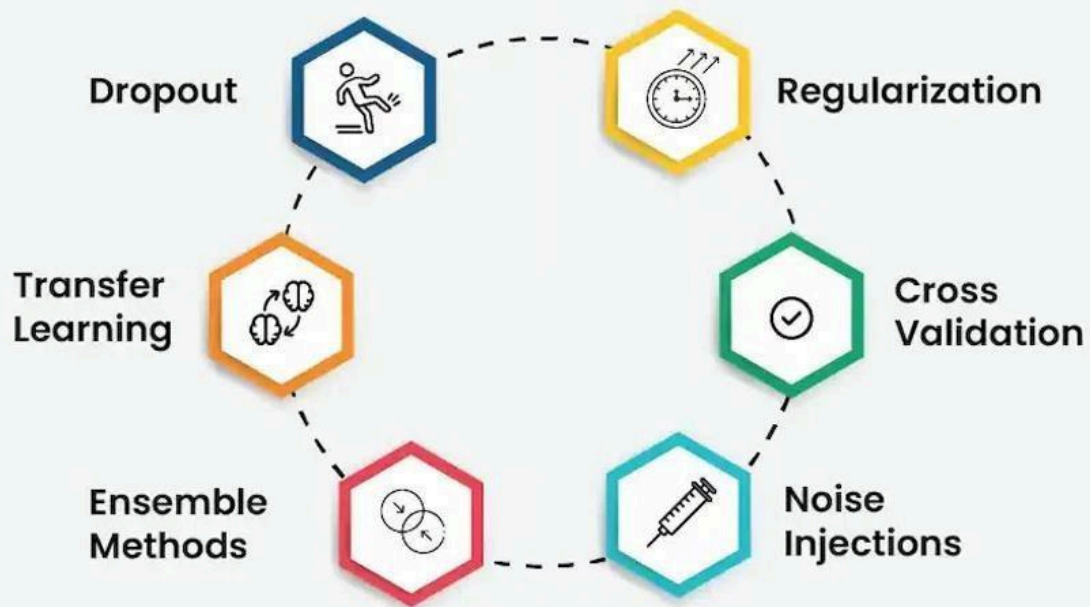


Generalization is a measure of how your model performs on predicting unseen data. So, it is important to come up with the best-generalized model to give better performance against future data. Let us first understand what is underfitting and overfitting, and then see what are the best practices to train a generalized model.



A: Underfitting, B: Generalized, C: Overfitting

Generalization Rules & Techniques in AI



Overfitting

Definition: Overfitting occurs when a model that performs well on the training data but poorly on unseen data because it has memorized specifics rather than generalizing the patterns.

Indicators:

- High accuracy on training data but low accuracy on validation/test data.

Causes:

- Complex models with too many parameters (e.g., deep neural networks).
- Small training datasets with insufficient diversity.

Solutions:

- **Regularization:** Techniques like L1, L2 regularization penalize large weights, encouraging simpler models.
- **Pruning:** Removing redundant parts of the model architecture, particularly for decision trees or neural networks.

- **Cross-validation:** Using techniques like k-fold cross-validation to better evaluate model performance.
- **Early Stopping:** Ending the training process when validation error starts increasing.
- **Ensembling:** Combining multiple models

Underfitting

Definition: Underfitting occurs when a model is too simplistic to capture the underlying structure of the data. It fails to fit the training data and, as a result, also performs poorly on new data.

- **Indicators:**
 - Low accuracy on both training and validation/test datasets.
- **Causes:**
 - Models that are too simple (e.g., linear regression for non-linear data).
 - Insufficient training time.
 - Lack of relevant features in the data.

Solutions:

- **Model Complexity:** Use more complex models that can capture patterns (e.g., neural networks, ensemble methods).
- **Feature Engineering:** Add or transform features to make patterns more accessible.
- **Increasing Training Time:** Train the model longer to allow it to learn complex patterns.
- **Hyperparameter Tuning:** Adjust parameters to allow for a more flexible fit to the data.
- **Increasing Data Quality/Quantity:** More or better-quality data can help the model learn more about the true data distribution.

Aspect	Overfitting	Underfitting
Model Fit	Fits too closely to the training data	Fails to fit the training data well
Training Accuracy	Very high	Low or moderate
Test Accuracy	Low	Low
Example Model	Deep neural network on small dataset	Linear regression for complex data
Solution	Regularization, simpler model, early stopping	More complex model, add features, tuning

